

Netty字符串编码器与解码器

北京理工大学计算机学院
金旭亮

最简单的编码器与解码器

流水线中的第一个解码器，一定是把从Channel中接收到的数据解码成ByteBuf类型，Netty为此提供了相应的基类—ByteToMessageDecoder，后继的解码器，可以从MessageToMessageDecoder派生，将ByteBuf中的数据转换为Java对象，或者把接收到的Java对象进行处理后，转换为另一种类型。

在很多的场景，Client与Server之间交换的数据，其实是字符串。为此，Netty提供了StringDecoder和StringEncoder两个组件，可以方便地将进站消息和出站消息解码或编码为字符串。

客户端代码

注意顺序

```
public class MyNettyClient {
    public static void main(String[] args) throws InterruptedException {
        NioEventLoopGroup workerGroup = new NioEventLoopGroup();
        Bootstrap bootstrap=new Bootstrap();
        bootstrap.group(workerGroup)
            .channel(NioSocketChannel.class)
            .handler(new ChannelInitializer<SocketChannel>() {
                @Override
                protected void initChannel(SocketChannel ch) throws Exception {
                    //先加入编码器
                    ch.pipeline().addLast(new StringEncoder(StandardCharsets.UTF_8));
                    //再加入解码器
                    ch.pipeline().addLast(new StringDecoder(StandardCharsets.UTF_8));
                    //添加自定义handler将数据发送到Server端
                    ch.pipeline().addLast(new MyClientSendHandler());
                }
            });
        var conn= bootstrap.connect("127.0.0.1",9000).addListener(future -> {
            if (future.isSuccess()) {
                System.out.println("连接成功");
            }else {
                System.out.println("连接失败");
            }
        });
        //阻塞等待通道的关闭
        conn.channel().closeFuture().sync();
        //关闭线程池，以结束整个进程
        workerGroup.shutdownGracefully().addListener(future -> {
            System.out.println("客户端已经退出");
        });
    }
}
```

客户端Handler

注意基类的泛型参数设定为String，则channelRead0方法的msg参数，也会“变”为String。

```
public class MyClientSendHandler extends SimpleChannelInboundHandler<String> {
    @Override
    public void channelActive(ChannelHandlerContext ctx) throws Exception {
        System.out.println(new Date()+":客户端发送数据");
        var messageData="你好, Netty Server".getBytes(StandardCharsets.UTF_8);
        //将数据发送到服务端
        ctx.writeAndFlush(Unpooled.wrappedBuffer(messageData));
    }
    //由于前一个入栈Handler已经将数据转换为了字符串，所以这里，直接就可以得到一个字符串格式的消息
    @Override
    protected void channelRead0(ChannelHandlerContext ctx, String msg) throws Exception {
        System.out.println("收到数据: "+msg);
        //收到数据之后，关闭连接
        ctx.close();
    }
}
```

服务端代码

```
public class MyNettyServer {
    private int port = 9000;

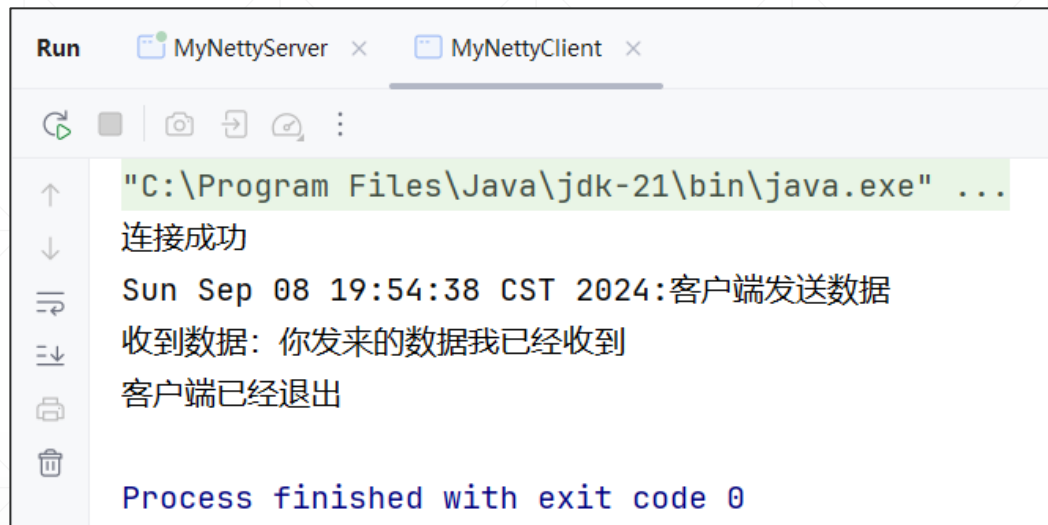
    public void run() throws InterruptedException {
        EventLoopGroup bossGroup = new NioEventLoopGroup();
        EventLoopGroup workerGroup = new NioEventLoopGroup();
        try {
            ServerBootstrap boot = new ServerBootstrap();
            boot.group(bossGroup, workerGroup)
                .channel(NioServerSocketChannel.class)
                .childHandler(new ChannelInitializer<SocketChannel>() {
                    @Override
                    protected void initChannel(SocketChannel socketChannel) throws Exception {
                        socketChannel.pipeline().addLast(new StringEncoder(StandardCharsets.UTF_8));
                        socketChannel.pipeline().addLast(new StringDecoder(StandardCharsets.UTF_8));
                        socketChannel.pipeline().addLast(new MyServerHandler());
                    }
                });
            ChannelFuture f = boot.bind(port).addListener(future -> {
                if (future.isSuccess()) {
                    System.out.println(port + "端口绑定成功.");
                } else {
                    System.out.println(port + "端口绑定成功失败");
                }
            });
            f.channel().closeFuture().sync();
        } finally {
            bossGroup.shutdownGracefully();
            workerGroup.shutdownGracefully();
        }
    }

    public static void main(String[] args) throws InterruptedException {
        new MyNettyServer().run();
    }
}
```

服务端Handler

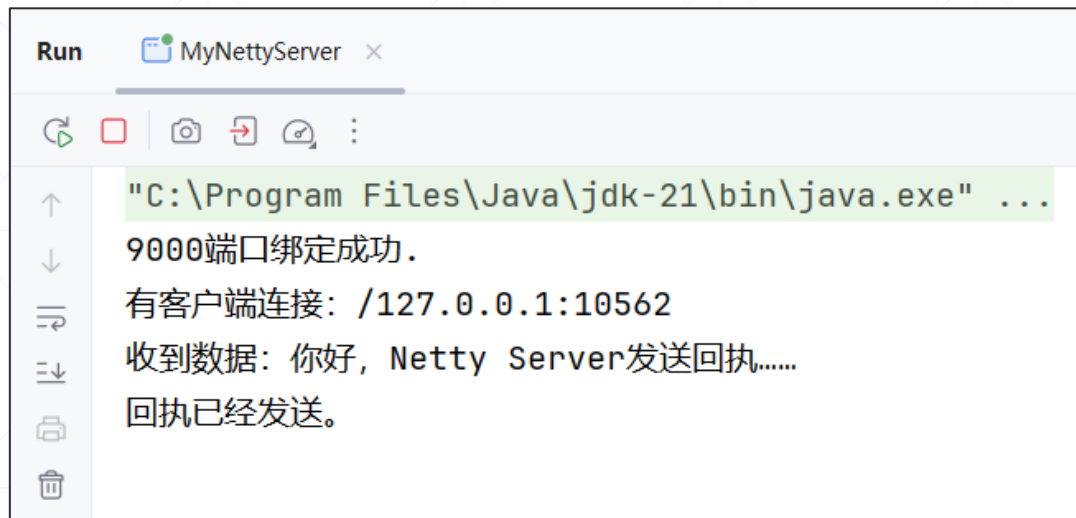
```
public class MyServerHandler extends SimpleChannelInboundHandler<String> {  
    @Override  
    public void channelActive(ChannelHandlerContext ctx) throws Exception {  
        System.out.println("有客户端连接: " + ctx.channel().remoteAddress());  
    }  
  
    @Override  
    protected void channelRead0(ChannelHandlerContext ctx, String msg) throws Exception {  
        System.out.println("收到数据: " + msg + "发送回执.....");  
        var sendBackMsg = "你发来的数据我已经收到".getBytes(StandardCharsets.UTF_8);  
        //这里发送的消息, 将会被StringEncoder编码为String, 再发送出去  
        ctx.writeAndFlush(Unpooled.wrappedBuffer(sendBackMsg));  
        System.out.println("回执已经发送。");  
    }  
}
```

运行截图



```
Run  MyNettyServer x  MyNettyClient x
"C:\Program Files\Java\jdk-21\bin\java.exe" ...
连接成功
Sun Sep 08 19:54:38 CST 2024:客户端发送数据
收到数据: 你发来的数据我已经收到
客户端已经退出

Process finished with exit code 0
```



```
Run  MyNettyServer x
"C:\Program Files\Java\jdk-21\bin\java.exe" ...
9000端口绑定成功.
有客户端连接: /127.0.0.1:10562
收到数据: 你好, Netty Server发送回执.....
回执已经发送。
```